

Experiment No. 2

Contrast Enhancement and Histogram-Based Image Processing using Python and OpenCV

Aim

To analyze and implement contrast enhancement techniques using histogram analysis and histogram equalization in Python and OpenCV for improving image quality and enhancing visual information in low-contrast images.

Purpose

Image contrast plays a significant role in computer vision systems. Poor illumination, sensor limitations, or environmental conditions can produce low-contrast images, making feature extraction and object recognition difficult.

In this experiment, contrast stretching, histogram analysis, Histogram Equalization, and Contrast Limited Adaptive Histogram Equalization (CLAHE) are implemented using Python and OpenCV. The enhanced results are compared using both images and histograms.

Prerequisites

- Basic knowledge of digital image processing
- Basic Python programming
- OpenCV, NumPy, and Matplotlib
- Understanding of grayscale images and pixel intensity distributions
- Google Colab / Jupyter Notebook / Visual Studio Code

```
!pip install opencv-python-headless -q

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from google.colab import files

print("Libraries imported successfully.")
```

Libraries imported successfully.

1. Load a Low-Contrast Image

```
uploaded = files.upload()

image_name = list(uploaded.keys())[0]
color_image = cv2.imread(image_name)

if color_image is None:
    raise ValueError("The uploaded file is not a valid image.")

gray_image = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)

print("Image loaded successfully!")
print("File name:", image_name)
print("Image size:", gray_image.shape[1], "x", gray_image.shape[0])
```

Choose Files 9011064.webp
9011064.webp(image/webp) - 205276 bytes, last modified: 10/3/2024 - 100% done
Saving 9011064.webp to 9011064 (1).webp
Image loaded successfully!
File name: 9011064 (1).webp
Image size: 3840 x 2160

2. Display Original Grayscale Image

```
plt.figure(figsize=(8, 6))
plt.imshow(gray_image, cmap="gray", vmin=0, vmax=255)
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

Original Grayscale Image

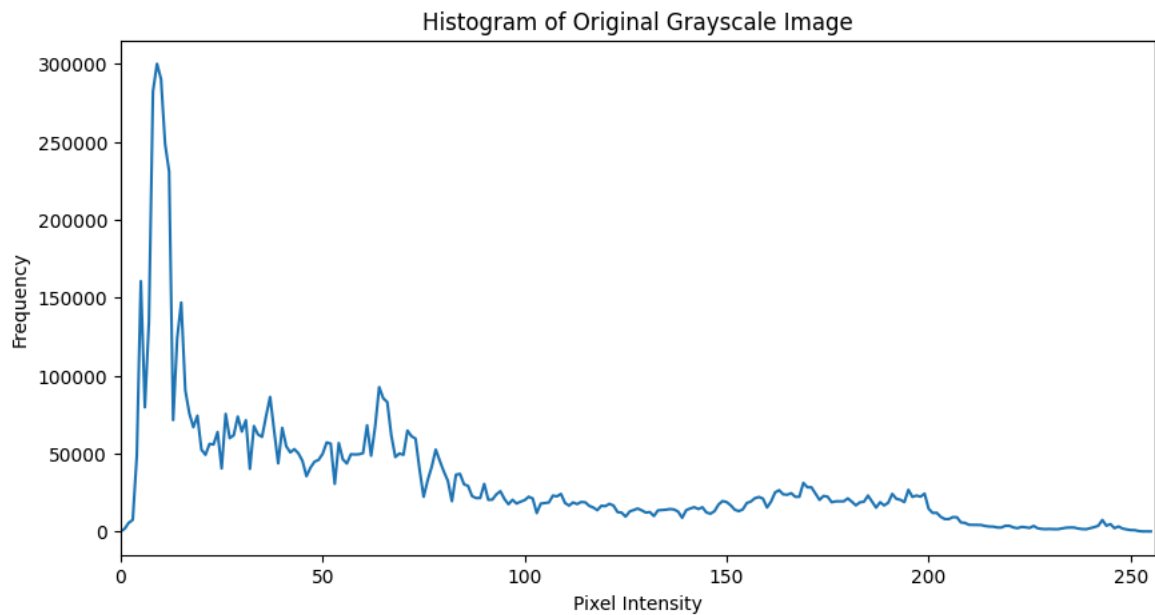


3. Generate Original Image Histogram

```
hist_original = cv2.calcHist([gray_image], [0], None, [256], [0, 256])

plt.figure(figsize=(10, 5))
plt.plot(hist_original)
plt.title("Histogram of Original Grayscale Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.show()

print("Minimum intensity:", gray_image.min())
print("Maximum intensity:", gray_image.max())
print("Mean intensity:", round(float(gray_image.mean()), 2))
print("Standard deviation:", round(float(gray_image.std()), 2))
```



Minimum intensity: 0
 Maximum intensity: 253
 Mean intensity: 66.92
 Standard deviation: 59.92

4. Contrast Stretching

```
min_value = int(gray_image.min())
max_value = int(gray_image.max())

if max_value > min_value:
    contrast_stretched = cv2.normalize(
        gray_image, None, 0, 255, cv2.NORM_MINMAX
    )
else:
    contrast_stretched = gray_image.copy()

plt.figure(figsize=(8, 6))
plt.imshow(contrast_stretched, cmap="gray", vmin=0, vmax=255)
plt.title("Contrast Stretched Image")
plt.axis("off")
plt.show()

print("Original intensity range:", min_value, "to", max_value)
print("Stretched intensity range:", contrast_stretched.min(), "to", contrast_stretched.max())
```

Contrast Stretched Image



Original intensity range: 0 to 253
Stretched intensity range: 0 to 255

5. Histogram Equalization

```
equalized_image = cv2.equalizeHist(gray_image)

plt.figure(figsize=(8, 6))
plt.imshow(equalized_image, cmap="gray", vmin=0, vmax=255)
plt.title("Histogram Equalized Image")
plt.axis("off")
plt.show()

print("Histogram Equalization applied successfully.")
```

Histogram Equalized Image



Histogram Equalization applied successfully.

6. Contrast Limited Adaptive Histogram Equalization (CLAHE)

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_image = clahe.apply(gray_image)

plt.figure(figsize=(8, 6))
plt.imshow(clahe_image, cmap="gray", vmin=0, vmax=255)
plt.title("CLAHE Enhanced Image")
plt.axis("off")
plt.show()
```

```
print("CLAHE applied successfully.")
print("Clip Limit: 2.0")
print("Tile Grid Size: 8 x 8")
```

CLAHE Enhanced Image



```
CLAHE applied successfully.
Clip Limit: 2.0
Tile Grid Size: 8 x 8
```

7. Visual Comparison of Enhancement Techniques

```
plt.figure(figsize=(16, 10))

plt.subplot(2, 2, 1)
plt.imshow(gray_image, cmap="gray", vmin=0, vmax=255)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(contrast_stretched, cmap="gray", vmin=0, vmax=255)
plt.title("Contrast Stretching")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(equalized_image, cmap="gray", vmin=0, vmax=255)
plt.title("Histogram Equalization")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(clahe_image, cmap="gray", vmin=0, vmax=255)
plt.title("CLAHE")
plt.axis("off")

plt.tight_layout()
plt.show()
```



8. Compare Histograms

```
hist_stretched = cv2.calcHist([contrast_stretched], [0], None, [256], [0, 256])
hist_equalized = cv2.calcHist([equalized_image], [0], None, [256], [0, 256])
hist_clahe = cv2.calcHist([clahe_image], [0], None, [256], [0, 256])
```

```
plt.figure(figsize=(14, 10))
```

```
plt.subplot(2, 2, 1)
plt.plot(hist_original)
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
```

```
plt.subplot(2, 2, 2)
plt.plot(hist_stretched)
plt.title("Contrast Stretching Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
```

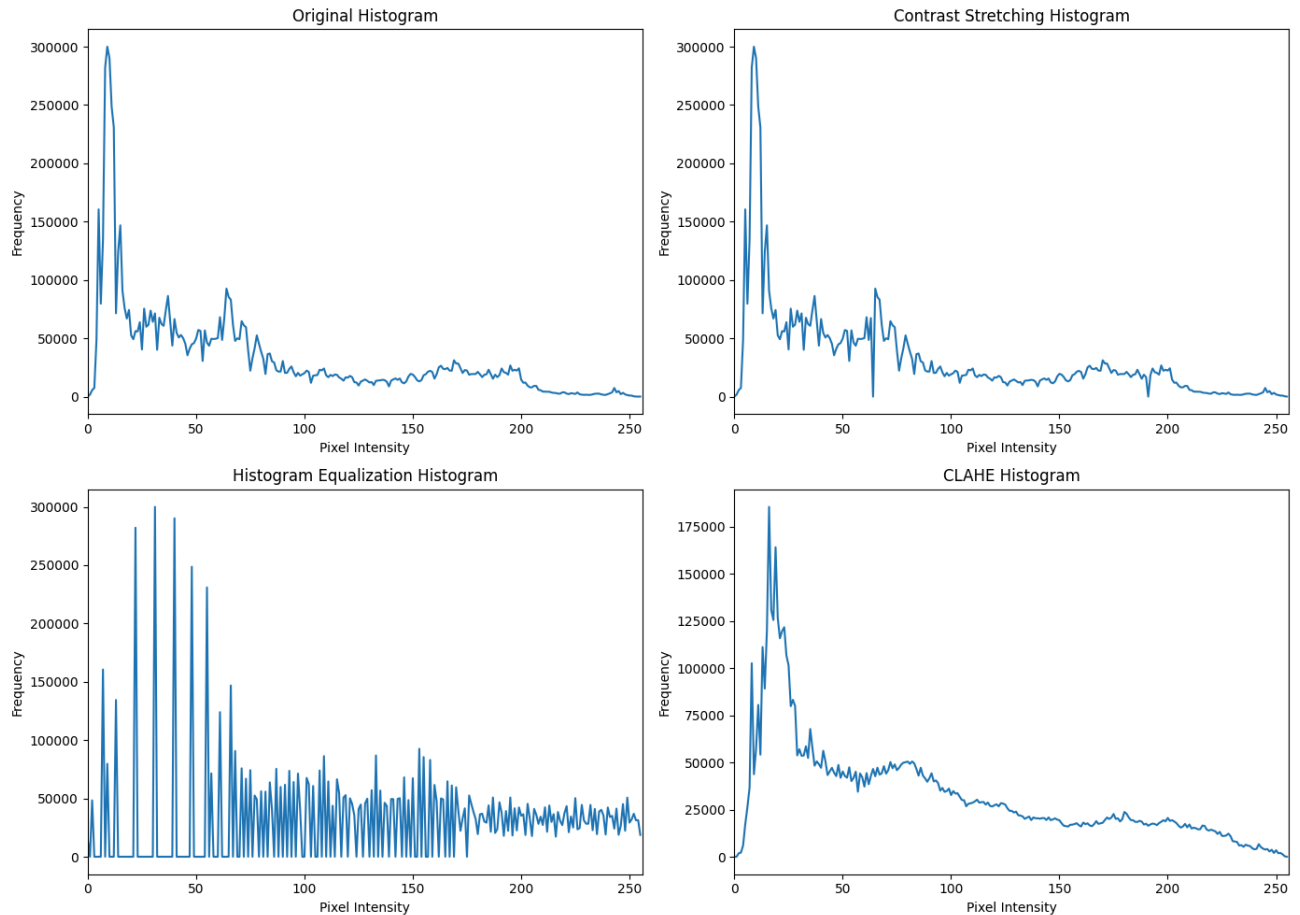
```

plt.subplot(2, 2, 3)
plt.plot(hist_equalized)
plt.title("Histogram Equalization Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])

plt.subplot(2, 2, 4)
plt.plot(hist_clahe)
plt.title("CLAHE Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])

plt.tight_layout()
plt.show()

```



9. Numerical Comparison

```
def image_statistics(name, img):
    print(name)
    print(" Minimum intensity:", int(img.min()))
    print(" Maximum intensity:", int(img.max()))
    print(" Mean intensity:", round(float(img.mean()), 2))
    print(" Standard deviation:", round(float(img.std()), 2))
    print()

image_statistics("Original", gray_image)
image_statistics("Contrast Stretching", contrast_stretched)
image_statistics("Histogram Equalization", equalized_image)
image_statistics("CLAHE", clahe_image)
```

```
Original
Minimum intensity: 0
Maximum intensity: 253
Mean intensity: 66.92
Standard deviation: 59.92

Contrast Stretching
Minimum intensity: 0
Maximum intensity: 255
Mean intensity: 67.39
Standard deviation: 60.45

Histogram Equalization
Minimum intensity: 0
Maximum intensity: 255
Mean intensity: 128.85
Standard deviation: 72.6

CLAHE
Minimum intensity: 0
Maximum intensity: 254
Mean intensity: 82.38
Standard deviation: 63.2
```

10. Save Enhanced Images

```
cv2.imwrite("original_grayscale.jpg", gray_image)
cv2.imwrite("contrast_stretched.jpg", contrast_stretched)
cv2.imwrite("histogram_equalized.jpg", equalized_image)
cv2.imwrite("clahe_enhanced.jpg", clahe_image)

print("All enhanced images saved successfully.")
```

All enhanced images saved successfully.

Observations

1. The original grayscale image and its intensity histogram were successfully obtained.
2. The histogram helped identify the distribution and range of pixel intensities.
3. Contrast stretching expanded the available intensity range and improved global contrast.
4. Histogram Equalization redistributed pixel intensities and enhanced overall contrast.
5. CLAHE improved local contrast by applying histogram equalization to small regions.
6. CLAHE limits excessive contrast amplification using a clip limit, reducing unwanted noise amplification.
7. The enhanced images showed improved visibility of details compared with the original image.
8. Histogram Equalization is effective for many images but can sometimes over-enhance noise or produce unnatural contrast.
9. CLAHE is generally more suitable when illumination varies across different regions of an image.
10. Histogram-based enhancement is an important preprocessing step for feature extraction, segmentation, object detection, and recognition.

Q1. 1. What is image contrast, and why is it important in image processing?

Answer: Image contrast is the difference in intensity or brightness between different regions of an image. Good contrast makes objects, edges, and details easier to distinguish. It is important because improved contrast can make visual information clearer and help later computer vision operations.

Q2. 2. Explain the concept of an image histogram. What information does it provide?

Answer: An image histogram is a graphical representation of the frequency of different pixel intensity values in an image. For an 8-bit grayscale image, it normally contains intensity levels from 0 to 255. It shows how pixels are distributed across dark, medium, and bright regions and helps analyze image brightness and contrast.

Q3. 3. Differentiate between Histogram Stretching and Histogram Equalization.

Answer: Histogram Stretching expands the existing intensity range, usually by mapping the minimum and maximum values to a desired range such as 0 to 255. Histogram Equalization redistributes intensity values using the cumulative distribution of the histogram to improve contrast. Stretching preserves the general distribution more directly, while equalization changes the distribution more substantially.

Q4. 4. What is Contrast Limited Adaptive Histogram Equalization (CLAHE)? How does it differ from standard Histogram Equalization?

Answer: CLAHE is a local contrast enhancement technique that divides an image into small tiles and performs histogram equalization within each tile. It also limits histogram amplification using a clip limit. Standard Histogram Equalization works globally on the entire image, while CLAHE adapts to local regions and is less likely to over-amplify noise.

Q5. 5. Why is histogram equalization commonly applied before feature extraction and image segmentation?

Answer: Histogram equalization can improve contrast and make edges, textures, and object boundaries more visible. This can provide feature extraction and segmentation algorithms with more distinguishable image information, which may improve their performance.

Q6. 6. Mention three real-world applications where histogram equalization is widely used.

Answer: Three applications are medical image enhancement, satellite and remote-sensing image enhancement, and low-light or surveillance image enhancement. It is also used in document processing, photography, and industrial inspection.

Q7. 7. What are the limitations of global histogram equalization?

Answer: Global histogram equalization uses the histogram of the entire image, so it may over-enhance some regions, amplify noise, reduce local detail, or produce unnatural-looking contrast. It may perform poorly when different parts of an image have very different illumination.

Q8. 8. How does contrast enhancement improve the performance of object detection and recognition systems?